MRes Artificial Intelligence

MRes Specialism: Academic Skills and Professional Development

Timileyin Ojo

2426478

Prof Celestine Iwendi

16-05-2025

Using Supervised Machine Learning to Predict the Bandgap of Inorganic Materials

Abstract

The study used supervised machine learning to predict the difference in the valence band and conduction band. The study goal is to train models that will learn from an existing inorganic dataset to predict the bandgap energy. The selected models used were Decision Tree, Random Forest (RF), XGBoost, and Support Vector Regression (SVR). These models were evaluated using metrics like Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score. Random Forest (RF) recorded 64% of the variation in the bandgap, closely with 62% from XGBoost and SVR has the poorest performance. From the top performance models, formation energy and volume were identified as the most important features for selection. The study indicated that Radom Forest is the best model to predict the bandgap energy of inorganic material and to validate the consistency of this outcome, 5-fold cross-validation was applied. While there is absence of environmental factor like temperature and pressure in the dataset, more research needs to be done to account for these features.

Background

There is a need to be able to predict the bandgap energy without using traditional methods like Density Functional Theory (DFT). Accurate prediction of bandgap energy in inorganic materials is important to the future of electronics; electronic devices need these features to thrive.

Bandgap determines the behaviour of a material whether it is a conductor, semi-conductor or an insulator for its usage in solar cells, diodes, transistors and LEDS (Zeb et al., 2024).

Density Functional Theory (DFT) can be very expensive and it lacks scalability with large materials, which might take days or weeks to execute (Varley, 2021).

To overcome this hurdle, supervised machine learning can be used to accurately predict the bandgap of large inorganic materials, which is accurate and faster (Stanley, Mayr and Gagliardi 2020; Shen et al., 2023).

Using Artificial Intelligence especially supervised machine learning in material science would solve complex material properties that traditional methods might find impossible (Nguyen et al., 2023). Models will be trained on numerous datasets of inorganic material and will be able to predict the bandgap of an unknown compound. By doing this the experimental process has been speed up, making it more effective (Park et al., 2020).

Aim

The aim of this study is to explore supervised machine learning models for bandgap prediction of inorganic materials.

Research Objectives

- To select features to use from the existing dataset.
- To identify features that significantly affect Bangap energy.
- To explore the best supervised machine learning model for bandgap prediction.
- To compare the metrics evaluation of the models using mean square error (MSE) and R^2 score.

Research Question

How can the bandgap energy of inorganic materials be accurately predicted by machine learning model based on their composition and structural properties?

Significance of the Study

This research makes use of machine learning to improve data driven material discovery. This helps to address the lapses of traditional computational techniques and enrich the ongoing attempts to expedite the advancement of materials for energy and electronic applications (Weng et al., 2016; Mishra et al., 2023). The outcome of this research would lead to the development of more eco-friendly and efficient semiconductors by identifying potential features with ideal bandgap values (Deng et al., 2024).

Literature Review

This literature review evaluates the application of S.E.N.D algorithms to predict the bandgap energy of inorganic materials through the use of machine learning model. The 'S' in S.E.N.D algorithm means similar work (A review of existing research paper that correspond), 'E' stands for exceptional works, that is, foremost research, 'N' for novel works -progress in the research and 'D' that stands for different work simply refers to alternative research paper.

The dependability and increasing adoption of machine learning to predict the bandgap of inorganic material is noted in multiple studies. With the use of random forests, support vector machines and gradient boosting, Zeb et al. (2024) conducted a comprehensive study on semiconductor bandgap prediction. From the investigated material dataset, they gathered features such as atomic radii, elementary composition and electro negativity. The gradient boosting demonstrated that machine learning can speed up bandgap discovery by achieving high predicted accuracy.

Comparably, the work of Stanley, Mayr and Gagliardi 2020) aimed at estimating the bandgaps and stability of lead-free perovskites for solar applications utilized a significant

data which revealed that decision tree-based methods yielded dependable outcomes during the selection of materials with stable and favourable bandgaps.

Also, Mishra et al. (2023) conducted a study in which they use machine learning to create broad bandgap perovskites for indoor solar applications. The study demonstrated the effectiveness of machine learning to forecast material for specific lightning situations by training models using elemental and structural features.

The research of Park et al. (2020) examined the structural deformation features in hybrid perovskites beyond elemental or crystal features. This remarkable research, which included these features, makes bandgap predictions much more accurate. The research demonstrates that the impact of expertise in a subject improve machine learning experience.

More so, Weng at all. (2016) conducted a notable research on bandgap. He used conceptual and experimental approach to establish the bandgap of Barium Bismuth Niobate double perovskites. They modified the atomic Composition and improved efficiency of water-splitting. They corroborated their findings through testing and DFT. This demonstrated the Influence of Chemistry and Computational methods.

Nguyen et al. (20233) employed data augmentation through Innovative method to improve anion sites in organic perovskites. They enhanced the prediction of machine learning and detected the augmented model generalisation by using multiple techniques to produce new data points through minor changes in anion sites. Nguyen et al. demonstrated that meticulous augmentation, a novel approach in machine learning may significantly improve a model's capacity for learning through machine learning models, Shem at al. (2023) developed 2D perovskites with customized bandgap. This innovative research evaluated multilayer designs with various electronic characteristics to predict bandgaps and guide the pursuit of enhanced solar cell performance. This new method shifts the emphasis to 2D materials, which are becoming more important in next generation of electronics.

In a different study, de Oliveira Neto et al. (2025) made use of Tutton salt crystal for thermal storage and UV-B filtration. Although their research did not focus primarily on

machine learning, it suggested that the bandgap could be influenced by a specific crystal structure made up of two different reasons. Their aim was to develop materials applicable beyond photovoltaics by showing the utility of bandgap information in various applications.

The analysis of Gao et al. (2021) in further research identified a thermally stable mixed material with an appropriate bandgap, in accordance with theoretical assumptions. They combined Cs with organic citations to produce a hybrid perovskite. This research contributes immensely to the body of work. It shows the effects of material engineering on bandgap and how it could be useful for machine learning models.

Methodology

This study employed a quantitative and exploratory analysis to predict bandgap energy of inorganic materials. This method of analysis was based on positivist philosophy which relies mostly on observable data and machine modelling than Interpretive reasoning. It utilized supervised regression techniques to predict material properties. It used numeral data from established Inorganic perovskite datasets (Stanley et al., 2020; Zeb et al., 2024).

The analysis relied on the ABX_3 Perovskite inorganic Compounds. Key features selected from the dataset Included formation enegy per atom, Energy above hill, density, Volume and crystal morphology (characterized by edge length and angles).

The main characteristics to figure out a material semiconducting behavior is bandgap energy, which was the main variable it is measured in electron volts (EV) (Weng et al., 2016). The process of removing features such as bulk and shear modules, rows that have missing values, and others that are not relevant is data cleaning.

Data cleaning made sure that only complete and essential observations were kept for modelling. Park et al. (2020) made use of this same method.

This study made use of four machine learning models: They are Decision tree Regressor, Random Forest Regressor, Support Vector Regressor and XGBoost Regressor. These

models were selected because they have shown to work well in material science and in predicting electronic properties (Nguyen et al., 2023; Shen et al., 2023).

While SVR was used to test a non-tree based learner, Random Forest and XGBoost are good at handling nonlinear and feature interaction. Scikit-learn and XGBoost libraries were used to train the models in Google colab. The training process involved 70% training data and 30% testing data with a fixed random state to make sure they were consistent.

Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the coefficient of the determination R^2 score are used to evaluate the performance of the model selected.

In order to validate the outcome of the finding, 5-fold cross-validation is also used, which is to provide consistency. This process aligns with previous studies on material prediction (Deng et al., Zeb et al., 2024).

While understanding the dataset, the feature importance plot visualizes the features that are most important for prediction. This aligns with previous studies from (Shen et al., 2023).

There is a scatter plot of predicted bandgap against actual bandgap, which compares both values and shows the accuracy of each model. Also, a bar chart of R^2 score is used to visualize each model's performance.

In the course of my work, I used Google Colab environment for all the coding, and analysed with Python tools like Pandas, Scikit-learn to train and evaluate models.

Result

In this study, I chose four different supervised learning models due to previous findings. The models that were used are Decision Tree (DT), Random Forest (RF), Support Vector Regression (SVR) and XGBoost. To evaluate the model performance, metrics like MAE, MSE and R^2 score were used.

From the table below, the column shows that Random Forest has MAE of 0.567, MSE of 0.794, RMSE of 0.891, and R^2 score of 0.643. This indicated that Random Forest had the best performance. With XGBoost having the second-highest performance, Mean Absolute Error (MAE) of 0.576967, Mean Squared Error (MSE) of 0.834194, Root Mean Squared Error (RMSE) of 0.913342, and R^2 score of 0.624487. Decision Tree with a fair R^2 Score of 0.299, and SVR performed poorly with the lowest score of 0.008.

Metric Comparison Table:

Metric	Decision Tree	Random Forest	SVR (RBF)	XGBoost
MAE	0.679702	0.567122	0.967593	0.576967
MSE	1.557757	0.794118	2.203904	0.834194
RMSE	1.248101	0.891133	1.484555	0.913342
R^2 Score	0.298774	0.642527	0.007910	0.624487

Table 1.

Figure 1 is the bar chart of the R^2 scores for all the models. The bar chart shows how each model performed in predicting the bandgap. Random Forest has the highest performance, followed by XGBoost. Both models performed better than Decision Tree and SVR. This validates the previous study that Random Forest and XGBoost are good at predicting bandgap.

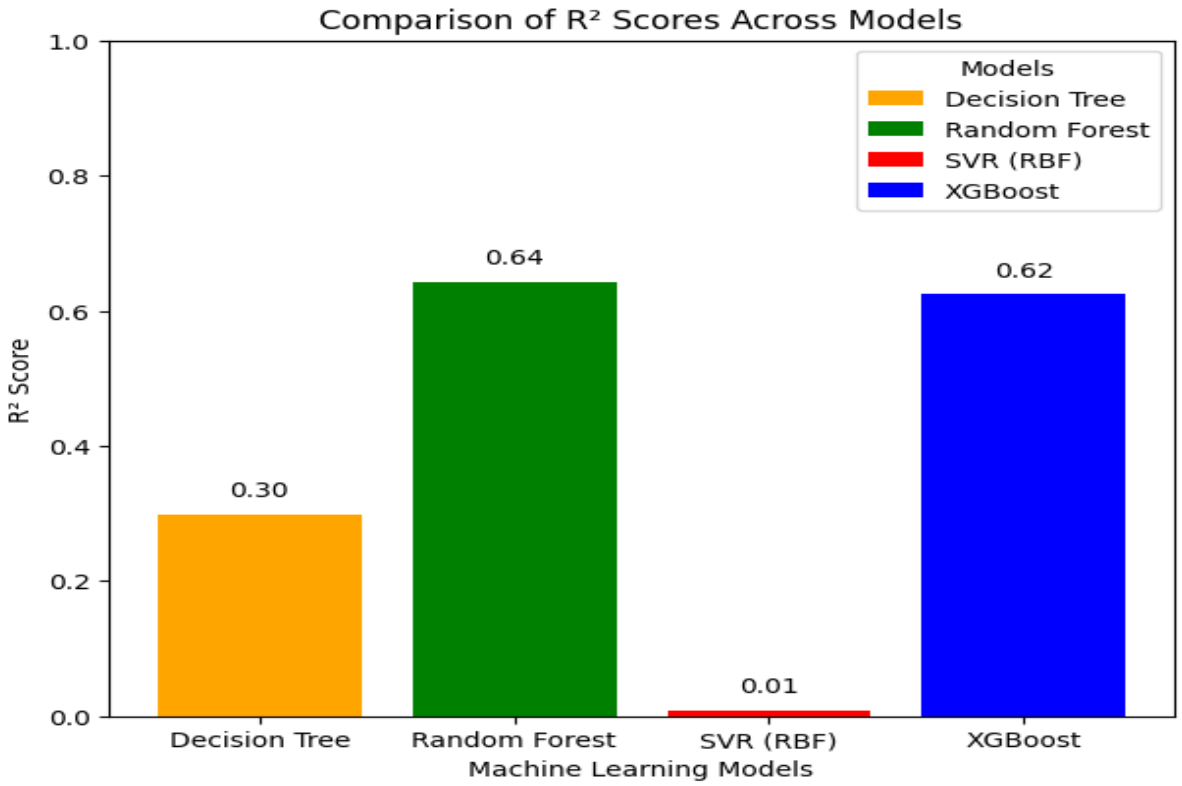


Fig 1.

Cross-validation was applied to determine if Random Forest was consistent; the dataset was divided into 5 different parts and tested 5 times. Figure 2 shows the R^2 score of each round and how their scores remained steady, the average part was close to the actual result. This means Random Forest gives a reliable outcome.

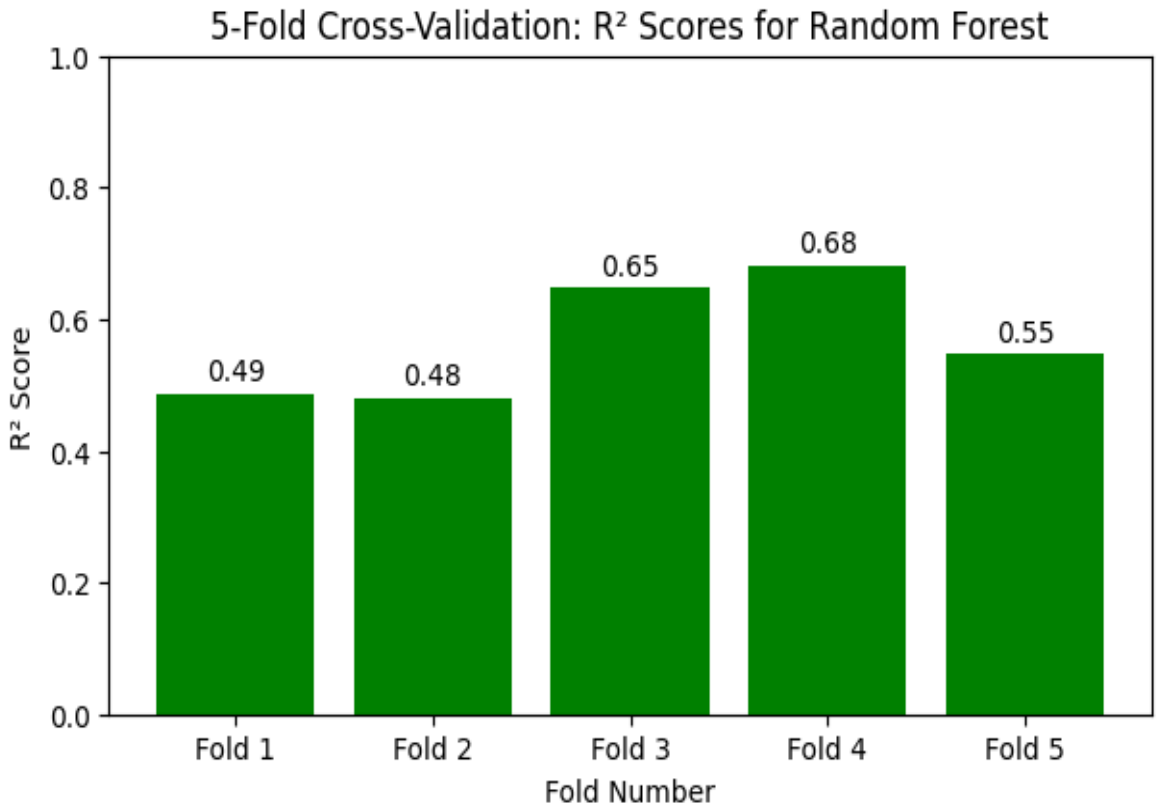


Fig 2.

To compare the predicted bandgap with the actual bandgap values, a scatter plot was used. Figures 3 and 4 show Random Forest and XGBoost scatter plots, respectively. In both plots, most points are close to the diagonal line. This means that the prediction was close to the actual values.

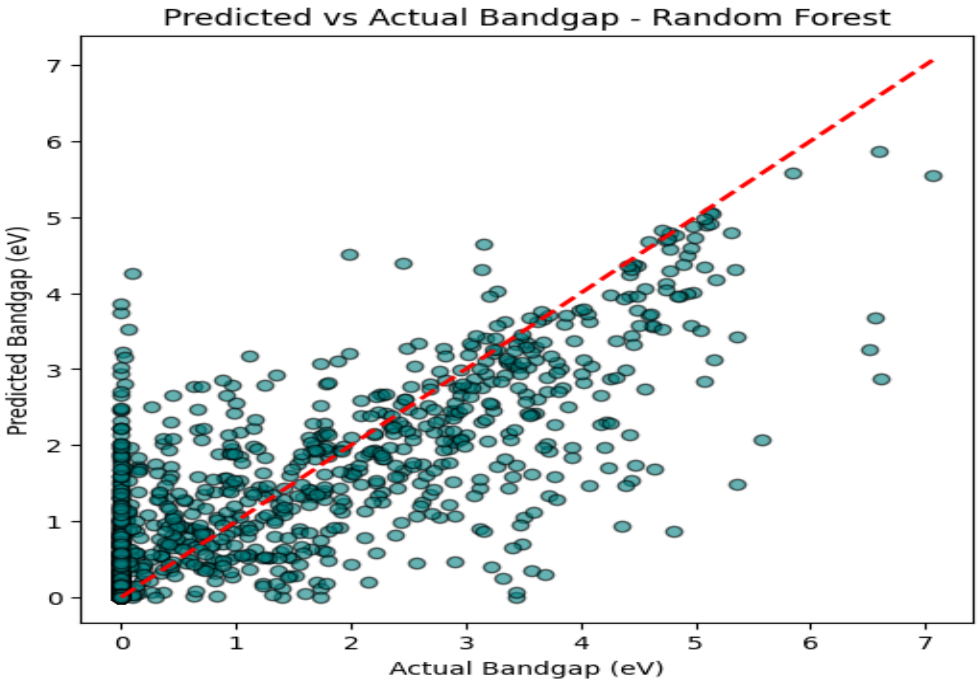

Predicted vs Actual Bandgap - Random Forest
Predicted Bandgap (eV)
Actual Bandgap (eV)
0
1
2
3
4
5
6
7

Fig 3.

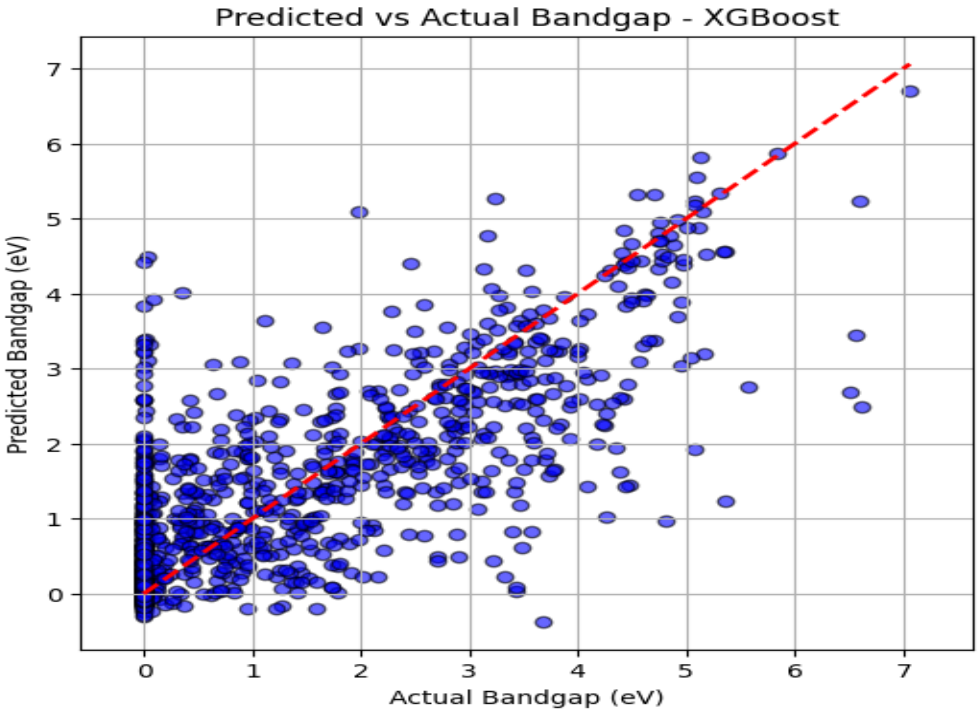

Predicted vs Actual Bandgap - XGBoost
Predicted Bandgap (eV)
Actual Bandgap (eV)
0
1
2
3
4
5
6
7

Fig 4.

Feature importance plots were created to show which of the features has more influence than the other. Figure 5 indicates the important features in Random Forest and Figure 6 shows the important features in XGBoost. For both models, formation energy, volume, and density were the most important features to consider.

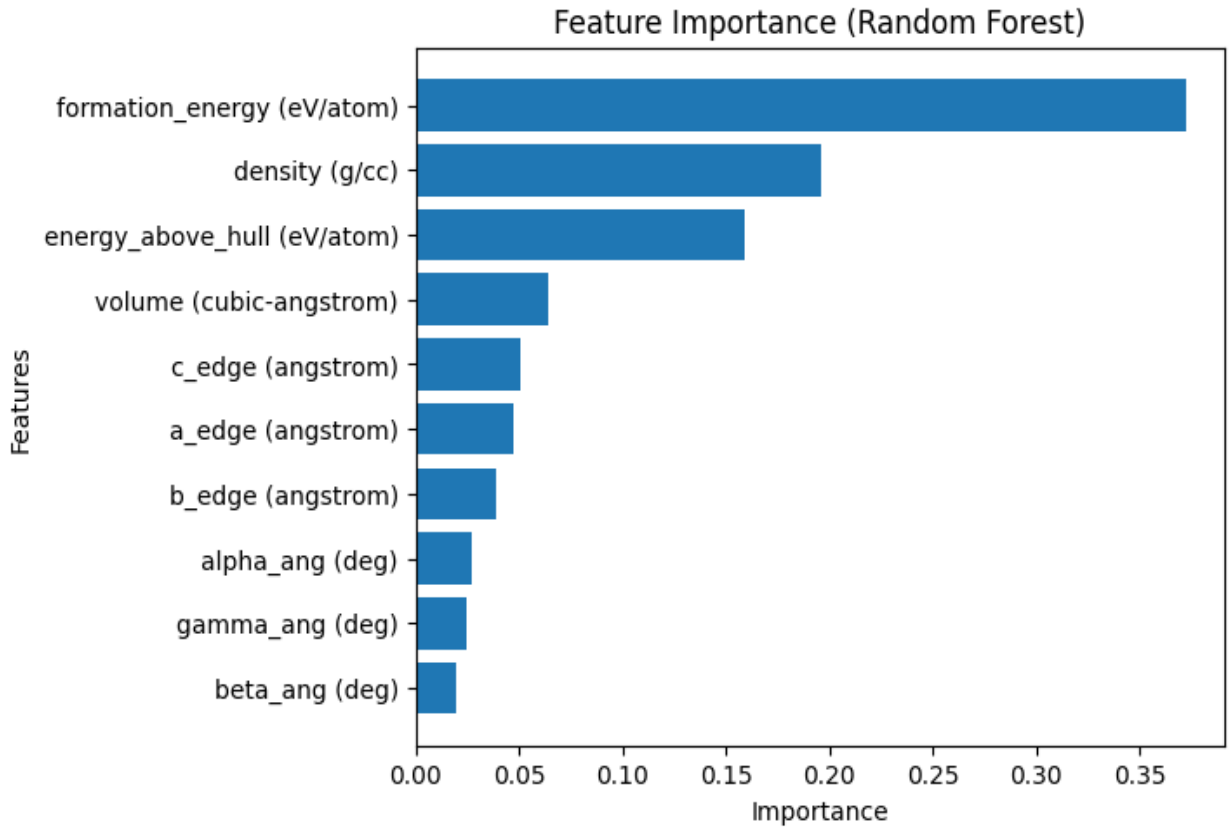


Fig 5.

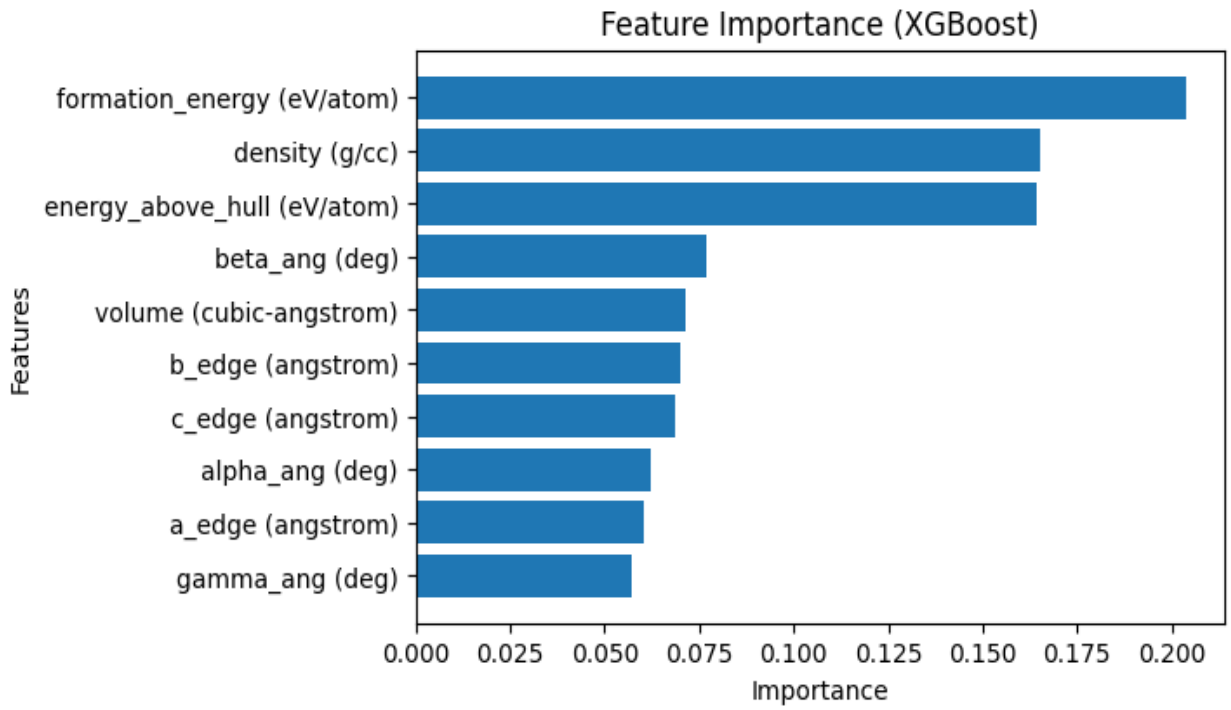


Fig 6.

Based on the above result, Random Forest is identified to be the best model for this research. Random Forest has the best performance and is the most accurate, followed by XGBoost. Decision Tree was fair, and SVR gave the poorest outcomes.

Discussion

This study shows that Random Forest and XGBoost, can accurately predict bandgap energy with structural features of inorganic materials, which is the same as earlier submissions of Zeb et. al (2024) and Shen et al. (2023). Their works showed that Random Forest and XGBoost impact materials science performance prediction. However, due to model limitations and real-world material behaviour that may affect the consistency of the result, their findings should be cautiously interpreted.

One major limitation is that the dataset does not recognize environmental faction like temperature, pressure, humidity and chemical exposure that can impact actual bandgap values in real-world scenarios. Research by Varley (2021) highlights the impact of thermal

stress on the optical and electrical properties of ultra-wide bandgap materials. Park et al. (2020) indicated that structural changes under stress can significantly change the bandgap properties of hybrid perovskites. They submitted that models may not perform optimally in real-world or industrial situations.

Predictive models are good to figure out structures, but cannot function well due to their inability to accommodate temperature and pressure. Further studies might provide more reliable models by involving dynamic environmental characteristics from simulations or experimental datasets.

The study show that Random Forest outperformed other models in terms of accuracy and reliability. XGBoost gave a comparable result despite its complex design. This finding is the same with Nguyen et al. (2023), which indicated that XGBoost, usually result declining returns without significant generalization improvements. Mishra et al. (2023) observed that decision tree's poor performance was due to overfitting. Scatter plot show the poor performance of the SVR model. This supports Stanley et al. (2020) submission that SVR struggle with high-dimensional material datasets.

Another subject of critical observation is feature importance. Formation energy, density, and lattice constants were found to be the most important predictors in Random Forest and XGBoost.

The result is the same with the concept of electronic band structure which is determined by lattice geometry, and crystal stability is reflected in formation energy (Weng et al., 2016; Gao et al., 2021).

Although environmental and compositional differences are not accounted for in this model, the feature significance plots validate the predictive capacity of structural descriptors. Methodologically, visual aids like predicted vs. actual graphs and R^2 score comparisons were important in pointing out the advantages and disadvantages of the model. For SVR, the predictions were scattered, which means underfitting, but Random Forest and XGBoost were packed closely in diagonal reducing the minimal error.

Roland et al. (2015) emphasised that there is a need to visualise the metric evaluation, which is the same as the research findings.

Conclusion

Supervised machine learning was used in this research to predict the bandgap of perovskite inorganic compounds. Four different supervised models were used, findings from this result indicated that Random Forest has the highest accuracy, accounting for the major variation in the bandgap. While both scatter plots and feature importance charts were used as a visual aid.

The major challenge the dataset used had was that it could not account for environmental features like temperature and pressure, among other natural factors. Which can affect how the material behaves in the real world. The model used might not be entirely reliable without these natural factors, even with the model's good performance on the dataset.

There is need to conduct more research with environmental features to help model stronger and suited for real world use. Even a simpler model like SVR can improve performance with reduced input features.

References

de Oliveira Neto, J.G., Valerio, A.R.P., da Silva, L.F.L., da Silva, L.M., Bordallo, H.N., de Sousa, F.F., dos Santos, A.O. and Lang, R. (2025) Design, characterization, and insights theoretical on $(NH_4)_2Fe_{0.11}Ni_{0.89}(SO_4)_2(H_2O)_6$ crystal: A novel Tutton salt for UV-B optical filters and thermochemical heat storage batteries. Journal of Solid State Chemistry, [Online] 347, pp. 125291. Available from: https://www.sciencedirect.com/journal/journal-of-solid-state-chemistry [Accessed 5 May 2025].

Deng, X., Zhang, Z., Zhang, Z., Wu, Y., Song, H., Li, H. and Luo, B. (2024) A prediction of all-inorganic lead-free halide perovskites for photovoltaic application: $Rb_3Mo_2Br_9$ and $Rb_3Mo_2Cl_9$. Advanced Science, [Online] 11(45), pp. e2407751-n/a. Available from: https://onlinelibrary.wiley.com/journal/21983844 [Accessed 5 May 2025].

Gao, Z., Wang, M., Zhang, H., Chen, S., Wu, C., Gates, I.D., Yang, W., Ding, X. and Yao, J. (2021) Design of $(C_3N_2H_5)(1-x)Cs_xPbI_3$ as a novel hybrid perovskite with strong stability and excellent photoelectric performance: A theoretical prediction. Solar Energy Materials and Solar Cells, 233, pp. 1.

Mishra, S., Boro, B., Bansal, N.K. and Singh, T. (2023) Machine learning-assisted design of wide bandgap perovskite materials for high-efficiency indoor photovoltaic applications. Materials Today Communications, 35, pp. 106376.

Murtaza, G., Ahmad, I. and Afaq, A. (2013) Shift of indirect to direct bandgap in going from K to Cs in $MCaF_3$ (M = K, Rb, Cs). Solid State Sciences, 16, pp. 152–157.

Nguyen, T., Kim, Y., Jung, I., Yang, O. and Akhtar, M.S. (2023) Exploring data augmentation and dimension reduction opportunities for predicting the bandgap of inorganic perovskite through anion site optimization. Photonics, 10(11), pp. 1232.

Park, H., Mall, R., Ali, A., Sanvito, S., Bensmail, H. and El-Mellouhi, F. (2020) Importance of structural deformation features in the prediction of hybrid perovskite bandgaps. Computational Materials Science, 184, pp. 109858.

Roland, S., Neubert, S., Albrecht, S., Stannowski, B., Seger, M., Facchetti, A., Schlatmann, R., Rech, B. and Neher, D. (2015) Hybrid organic/inorganic thin-film

multijunction solar cells exceeding 11% power conversion efficiency. Advanced Materials (Weinheim), 27(7), pp. 1262–1267.

Shen, Y., Wang, J., Ji, X. and Lu, W. (2023) Machine learning-assisted discovery of 2D perovskites with tailored bandgap for solar cells. Advanced Theory and Simulations, [Online] 6(6), pp. n/a. Available from: https://onlinelibrary.wiley.com/journal/25130358 [Accessed 6 May 2025].

Stanley, J.C., Mayr, F. and Gagliardi, A. (2020) Machine learning stability and bandgaps of lead-free perovskites for photovoltaics. Advanced Theory and Simulations, [Online] 3(1), pp. n/a. Available from: https://onlinelibrary.wiley.com/journal/25130358 [Accessed 7 May 2025].

Varley, J.B. (2021) First-principles calculations of structural, electrical, and optical properties of ultra-wide bandgap $(Al_xGa_{1-x})_2O_3$ alloys. Journal of Materials Research, 36(23), pp. 4790–4803.

Weng, B., Xiao, Z., Meng, W., Grice, C.R., Poudel, T., Deng, X., Yan, Y., Univ. of California, Oakland, CA (United States) and Lawrence Berkeley National Laboratory (LBNL), Berkeley, CA (United States). National Energy Research Scientific Computing Center (NERSC) (2016) Bandgap engineering of barium bismuth niobate double perovskite for photoelectrochemical water oxidation. Advanced Energy Materials, [Online] 7(9). Available from: https://onlinelibrary.wiley.com/journal/16146840 [Accessed 9 May 2025].

Zeb, M.H., Rehman, A., Siddiqah, M., Bao, Q., Shabbir, B. and Kabir, M.Z. (2024) Machine learning-enhanced prediction of inorganic semiconductor bandgaps for advancing optoelectronic technologies. Advanced Theory and Simulations, [Online] 7(7), pp. n/a. Available from: https://onlinelibrary.wiley.com/journal/25130358 [Accessed 8 May 2025].

```
#Upload the ABX₃ Perovskite dataset


#Import library functions

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


#Load ABX₃ Perovskite dataset

df = pd.read_csv("/content/abc dataset.csv")

df
```

omposition	a_edge (angstrom)	b_edge (angstrom)	c_edge (angstrom)	alpha_ang (deg)	beta_ang (deg)	gamma_ang (deg)	...	energy_per_atom (eV/atom)
Ac1 Al1 O3	3.858634	3.858634	3.858634	90.000000	90.0	90.000000	...	-8.232146
Ac1 B1 O3	3.721668	3.721668	3.721668	90.000000	90.0	90.000000	...	-7.604280
Ac1 Cr1 O3	3.944287	3.944302	3.944272	90.000000	90.0	90.000000	...	-8.862593
Ac1 Cu1 O3	3.913331	3.913331	3.913331	90.000000	90.0	90.000000	...	-7.035745
Ac1 Fe1 O3	3.953570	3.953568	3.953585	90.000092	90.0	90.000000	...	-8.258555
...	...	...	...	...	...	...	...	...
Nd1 Mn1 O3	14.392943	14.078227	19.909366	87.880792	90.0	90.000000	...	-4.747233
Nd1 Mn1 O3	5.499704	5.657811	7.900171	89.429250	90.0	90.000000	...	-8.706946
Nd1 Mn1 O3	5.439007	5.594976	7.743939	90.000000	90.0	90.000000	...	-8.718288
Nd1 Ni1 Ge3	11.049211	11.049211	4.170587	90.000000	90.0	158.488336	...	-5.481031
Nd1 Ni1 O3	5.383764	5.388578	7.604511	90.000000	90.0	90.000000	...	-7.503700

```
#Clean dataset
#Drop irrelevant columns and remove missing values

df_clean = df.drop(columns=['bulk_modulus (GPa)', 'shear_modulus (GPa)']).dropna()

df_clean


#Define features to use for X and Y
#Select features and target to use from ABX₃ Perovskite dataset

features = [
    'formation_energy (eV/atom)', 'energy_above_hull (eV/atom)',
    'density (g/cc)', 'volume (cubic-angstrom)',
    'a_edge (angstrom)', 'b_edge (angstrom)', 'c_edge (angstrom)',
    'alpha_ang (deg)', 'beta_ang (deg)', 'gamma_ang (deg)'
]
target = 'band_gap (eV)'

X = df_clean[features].dropna()

y = df_clean[target]
```

```
y = y[X.index]  # Align y with cleaned X


# Train/Test Split
#Define 70% Training and 30% test for X and y.

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)


#Decision Tree Regression
#Train DT model

model = DecisionTreeRegressor()

model.fit(X_train, y_train)
```

```
DecisionTreeRegressor
DecisionTreeRegressor()
```

```
#Create a model to predict test data

y_predict = model.predict(X_test)


#Using DT to evaluate metrics

print("DECISION TREE RESULTS")

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_predict))

print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_predict))

print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test, y_predict)))

print("R² Score:", r2_score(y_test, y_predict))
```

```
DECISION TREE RESULTS
Mean Absolute Error (MAE): 0.6797019751280174
Mean Squared Error (MSE): 1.5577572742648134
Root Mean Squared Error (RMSE): 1.2481014679363267
R² Score: 0.29877358942307863
```

```
#Random Forest Regression
#Train RF model

model = RandomForestRegressor(n_estimators=200, random_state=1)

model.fit(X_train, y_train)
```

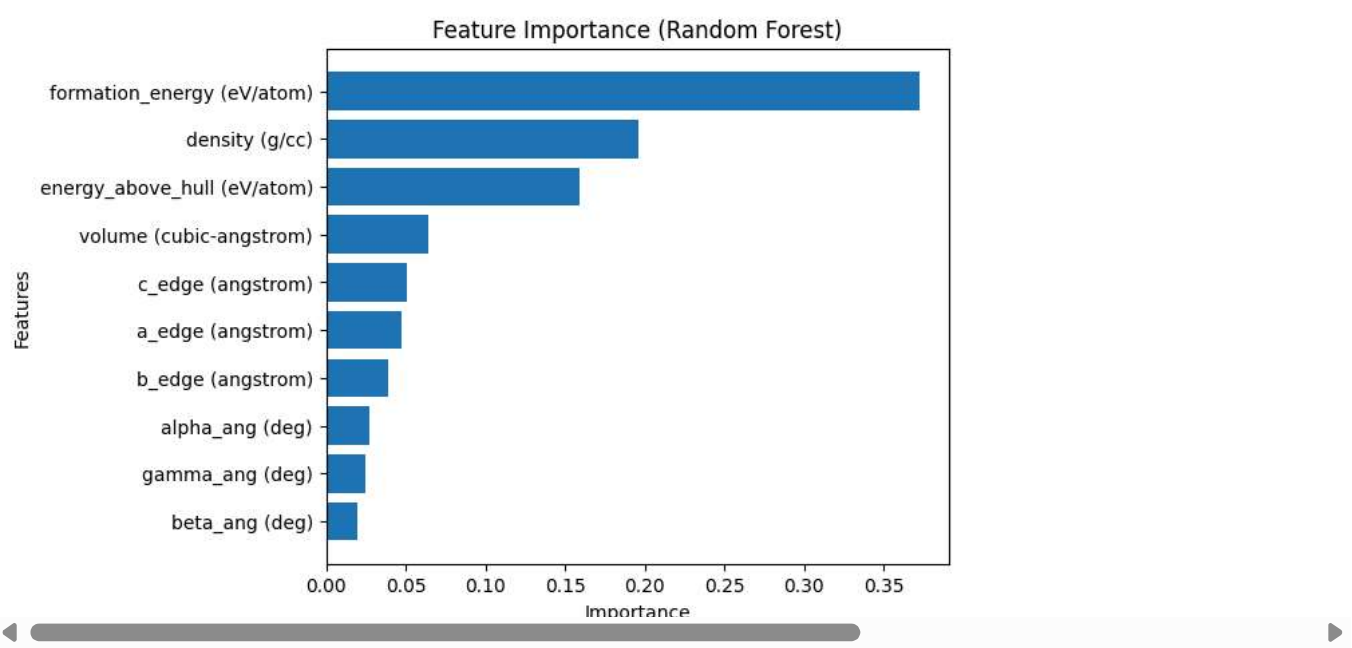
```
RandomForestRegressor
RandomForestRegressor(n_estimators=200, random_state=1)
```

```
#Create a model to predict test data

y_predict = model.predict(X_test)


#Using RF to evaluate metrics

print("RANDOM FOREST RESULTS")

print("Mean Absolute Error:", mean_absolute_error(y_test, y_predict))

print("Mean Squared Error:", mean_squared_error(y_test, y_predict))

print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_predict)))

print("R² Score:", r2_score(y_test, y_predict))
```

```
RANDOM FOREST RESULTS
Mean Absolute Error: 0.567121591441112
Mean Squared Error: 0.7941178309441216
Root Mean Squared Error: 0.8911328918540273
R² Score: 0.6425268523102315
```

```
#Plot feature importance for Random Forest

importances = model.feature_importances_ # Get the importance score of each feature from the trained Random Forest model
```

```
feature_names = X.columns # Store the feature names (column names) for labelling the plot

indices = np.argsort(importances)[::-1] # Sort feature indices by importance in descending order (most important first)

plt.figure(figsize=(6, 5))
plt.barh(range(len(importances)), importances[indices]) # Draw a horizontal bar chart using sorted importances

plt.yticks(range(len(importances)), feature_names[indices]) # Label each bar with the corresponding feature name

plt.xlabel("Importance")
plt.ylabel("Features")
plt.title("Feature Importance (Random Forest)")

plt.gca().invert_yaxis() # Flip the y-axis so the most important feature appears at the top

plt.show()
```




```
#Support Vector Regressor (RBF Kernel)
#Train SVR model

model = SVR(kernel='rbf')

model.fit(X_train, y_train)
```

```
SVR
SVR()
```

```
#Create a model to predict test data

y_predict = model.predict(X_test)


#Using SVR to evaluate metrics

print("SUPPORT VECTOR REGRESSOR (SVR) RESULTS")

print("Mean Absolute Error:", mean_absolute_error(y_test, y_predict))

print("Mean Squared Error:", mean_squared_error(y_test, y_predict))

print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_predict)))

print("R² Score:", r2_score(y_test, y_predict))
```

```
SUPPORT VECTOR REGRESSOR (SVR) RESULTS
Mean Absolute Error: 0.9675933493181171
Mean Squared Error: 2.203903864578048
Root Mean Squared Error: 1.4845551066154627
R² Score: 0.007909883172241061
```

```
#Extreme Gradient Boosting Regression
#Train XGB model

from xgboost import XGBRegressor
```

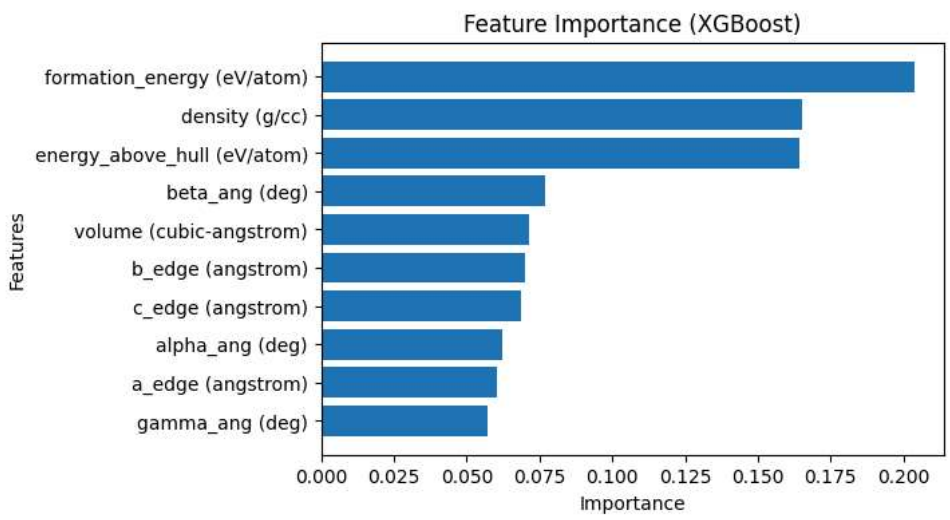
```
model = XGBRegressor(n_estimators=300, learning_rate=0.1, random_state=None)

model.fit(X_train, y_train)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=0.1, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=None, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=300, n_jobs=None,
             num_parallel_tree=None, random_state=None, ...)
```

```
#Create a model to predict test data

y_predict = model.predict(X_test)


#Using XGBoost to evaluate metrics

print("XGBOOST RESULTS")

print("Mean Absolute Error:", mean_absolute_error(y_test, y_predict))

print("Mean Squared Error:", mean_squared_error(y_test, y_predict))

print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_predict)))

print("R² Score:", r2_score(y_test, y_predict))
```

```
XGBOOST RESULTS
Mean Absolute Error: 0.5769672320565657
Mean Squared Error: 0.8341938895033694
Root Mean Squared Error: 0.9133421535784765
R² Score: 0.6244865637762975
```

```
#Feature Importance Plot for XGBoost

importances = model.feature_importances_
feature_names = X.columns
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(6, 4))
plt.barh(range(len(importances)), importances[indices])
plt.yticks(range(len(importances)), feature_names[indices])

plt.xlabel("Importance")
plt.ylabel("Features")
plt.title("Feature Importance (XGBoost)")
plt.gca().invert_yaxis()

plt.show()  # Display the plot
```



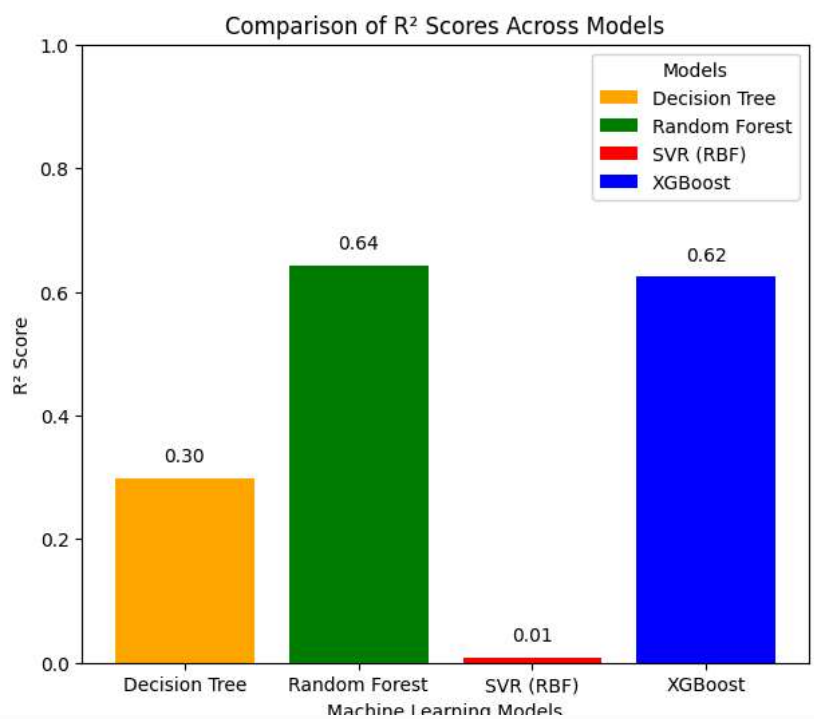
```
# Define evaluation metrics for each model

metrics_data = {
    'Metric': ['MAE', 'MSE', 'RMSE', 'R² Score'],
    'Decision Tree': [0.6797019751280174, 1.5577572742648134, 1.2481014679363267, 0.29877358942307863],
    'Random Forest': [0.567121591441112, 0.7941178309441216, 0.8911328918540273, 0.6425268523102315],
    'SVR (RBF)': [0.9675933493181171, 2.203903864578048, 1.4845551066154627, 0.007909883172241061],
    'XGBoost': [0.5769672320565657, 0.8341938895033694, 0.9133421535784765, 0.6244865637762975]
}

# Create the comparison DataFrame
comparison_df = pd.DataFrame(metrics_data)

print("Model Comparison Table:")

comparison_df
```

```
Model Comparison Table:
```

	Metric	Decision Tree	Random Forest	SVR (RBF)	XGBoost
0	MAE	0.679702	0.567122	0.967593	0.576967
1	MSE	1.557757	0.794118	2.203904	0.834194
2	RMSE	1.248101	0.891133	1.484555	0.913342
3	R² Score	0.298774	0.642527	0.007910	0.624487

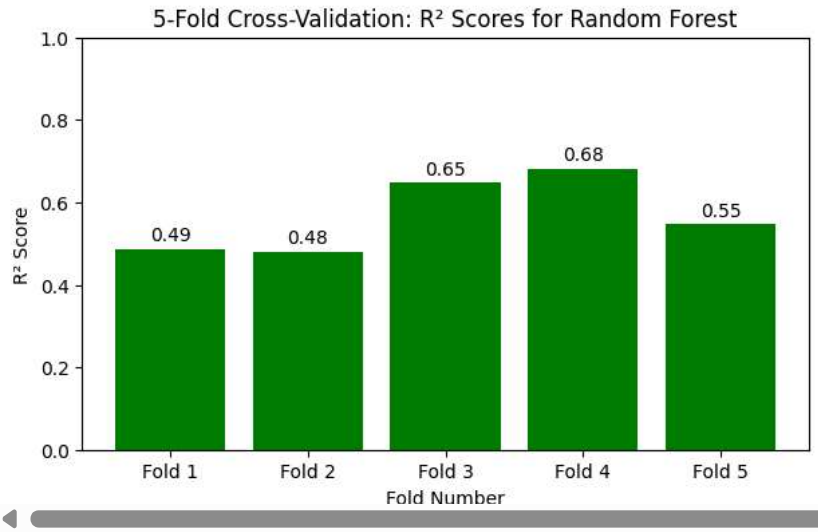
Next steps: Generate code with comparison_df | View recommended plots | New interactive sheet

```
# Define model names and their corresponding R² scores

model_names = ['Decision Tree', 'Random Forest', 'SVR (RBF)', 'XGBoost']

r2_scores = [0.298774, 0.642527, 0.007910, 0.624487]  # Replace with exact values if needed

# Define custom colours for each model
colors = ['orange', 'green', 'red', 'blue']

# Create Bar Chart
plt.figure(figsize=(7, 6))
bars = plt.bar(model_names, r2_scores, color=colors)

plt.xlabel('Machine Learning Models')
plt.ylabel('R² Score')
plt.title('Comparison of R² Scores Across Models')
plt.ylim(0, 1)
#plt.grid(axis='y')

# Add value labels on top of bars
for bar, score in zip(bars, r2_scores):
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, yval + 0.02, f'{yval:.2f}', ha='center', va='bottom')

plt.legend(bars, model_names, title='Models')

plt.show()
```

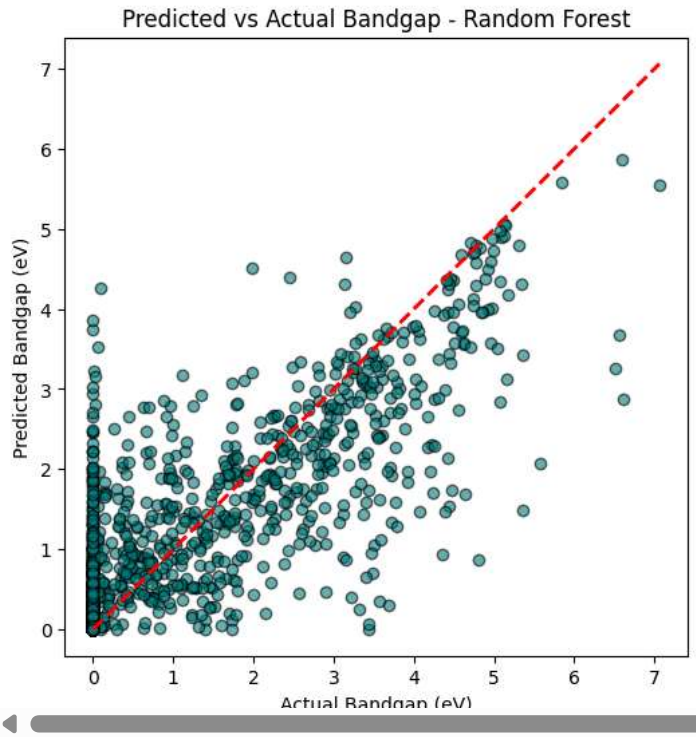



```
# Define the Random Forest model

from sklearn.model_selection import cross_val_score

model = RandomForestRegressor(n_estimators=200, random_state=1)

# Perform 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5, scoring='r2')

print("RANDOM FOREST CROSS-VALIDATION (R² Score)")

print("Fold R² Scores:", np.round(scores, 4))

print("Average R² Score:", scores.mean())

# Plot the R² scores
plt.figure(figsize=(7, 4))
plt.bar(range(1, 6), scores, color='green')
plt.ylim(0, 1)
plt.xticks(range(1, 6), labels=[f'Fold {i}' for i in range(1, 6)])

plt.ylabel("R² Score")
plt.xlabel("Fold Number")
plt.title("5-Fold Cross-Validation: R² Scores for Random Forest")
#plt.grid(axis='y')

# Annotate bar values
for i, score in enumerate(scores):
    plt.text(i + 1, score + 0.02, f"{score:.2f}", ha='center')

plt.show()
```

```
RANDOM FOREST CROSS-VALIDATION (R² Score)
Fold R² Scores: [0.4872 0.4797 0.6479 0.6817 0.5467]
Average R² Score: 0.5686286654982395
```




```
# Predict using the Random Forest model
model = RandomForestRegressor(n_estimators=200, random_state=1)
model.fit(X_train, y_train)
y_predict = model.predict(X_test)

# Create the scatter plot
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_predict, alpha=0.6, color='teal', edgecolors='k')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', linewidth=2)

plt.xlabel("Actual Bandgap (eV)")
plt.ylabel("Predicted Bandgap (eV)")
plt.title("Predicted vs Actual Bandgap - Random Forest")
#plt.grid(True)
plt.axis('equal')

plt.show()
```




```
#Predict using XGBoost model
model = XGBRegressor(n_estimators=300, learning_rate=0.1, random_state=None)
model.fit(X_train, y_train)
y_predict = model.predict(X_test)

plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_predict, alpha=0.6, color='blue', edgecolors='k')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', linewidth=2)
plt.xlabel("Actual Bandgap (eV)")
```

```
plt.ylabel("Predicted Bandgap (eV)")
plt.title("Predicted vs Actual Bandgap - XGBoost")
plt.grid(True)
plt.axis('equal')

plt.show()
```